

AuraCredit AI — Project Report

Point-by-Point Technical Implementation & Architecture Documentation

1. Executive Summary

AuraCredit AI is an end-to-end, high-aesthetic machine learning application designed to assess consumer credit risk and predict individual creditworthiness. Built using Python, Flask, Scikit-Learn, and Chart.js, the system provides credit analysts with a visual playground to tune model parameters, view training results, inspect datasets, and perform real-time client evaluations.

2. Detailed Project Components

The project consists of the following modular components:

- **Machine Learning Engine (`model_pipeline.py`):** Contains the core pipeline logic for generating realistic synthetic data (1,000 cases), cleaning raw variables, scaling numeric features, performing train-test splitting, training multiple algorithms (Logistic Regression, Decision Trees, and Random Forests), computing classification metrics, and generating ROC curve coordinates.
- **Flask API Backend (`app.py`):** Integrates the ML model with the web front-end. It runs the models automatically on startup, stores parameters, and exposes API endpoints for data previews, retraining requests, and individual credit risk scoring.
- **Dashboard Front-End (`templates/index.html`):** A semantic layout built with HTML5, containing sidebar hyperparameter inputs (sliders), summary metric cards, interactive canvas zones for charts, applicant database tables, and the applicant assessment form.
- **Premium Stylesheet (`static/css/style.css`):** Implements a modern dark-glassmorphic style featuring backdrop-filter blurring, subtle indigo-to-violet linear gradients, responsive card positioning, and dynamic hover feedback animations. Support for light/dark theme switching is fully implemented using variable sheets.
- **Interactive Controller (`static/js/main.js`):** Interacts with server endpoints, switches view tabs, updates circular credit score gradients, and updates visual charts dynamically.
- **Programmatic Verification Suites (`test_pipeline.py` & `verify_endpoints.py`):** Unit tests asserting the algorithmic accuracy and integration tests confirming response payloads of web endpoints.

3. Feature Engineering Details

To improve classifier predictive power, the system automatically translates raw financial metrics into key ratios:

- **Credit Utilization Ratio:** Current outstanding balance divided by the total credit limit. Ratios above 60% indicate higher borrowing strain.
- **Debt-to-Income (DTI) Ratio:** Annualized monthly debt payments divided by annual income. Ratios above 40% suggest repayment risk.
- **Payment-to-Income Ratio:** Monthly debt payments relative to monthly income.
- **Home Ownership Coding:** One-hot encoding mapping 'Own', 'Mortgage', and 'Rent' statuses to binary columns.

4. Models & Algorithms Implemented

Three distinct machine learning models are trained and compared in parallel:

- 1. Logistic Regression:** A linear baseline model that maps a combination of scaled inputs to a sigmoid probability. Offers fast, interpretable coefficient analysis.
- 2. Decision Tree Classifier:** A tree-structured model that segments applicants using simple feature rules. Offers structural interpretability.
- 3. Random Forest Classifier:** An ensemble of decision trees. Averaging outputs reduces variance, offering robust predictive accuracy (typically the champion model).

5. Technical Stack & Resources

Technology	Role / Usage	Key Packages
Python 3.14	Backend Computing & Modelling	Scikit-Learn, Pandas, NumPy, Joblib
Flask	Web Framework & API Server	Werkzeug, Jinja2, Click
HTML5 / JavaScript	Frontend Interface Logic	Chart.js (v4), FontAwesome (v6)
Vanilla CSS3	Visual Presentation	Glassmorphism variables, flexbox/grid

6. Programmatic Verification Results

The system has been successfully verified via a two-layer automated testing environment:

Layer A: Unit Testing (test_pipeline.py): Verifies individual data generation columns, features engineering formulas, cross-validation model structures, saving/loading, and single-applicant predictions. Results: **5/5 Tests Passed.**

Layer B: Endpoint Testing (verify_endpoints.py): Tests the live API. Verifies that GET / serves HTML, /api/data delivers 50 data records, /api/train runs retraining smoothly, and /api/predict correctly identifies low-risk approvals (95% score) vs high-risk rejections (0% score). Results: **All API routes verified successfully.**

7. Local Running Instructions

The Flask application is running as a background task. Follow these instructions to view the live dashboard:

- 1. Open your internet browser.
- 2. Navigate to: **http://127.0.0.1:5000/**
- 3. Use the left sidebar to tweak hyperparameters, click 'Train models', view metric graphs, and input client profiles to test outcomes.

8. Project Summary

In summary, the AuraCredit AI project delivers a fully integrated, scalable credit assessment solution. By transforming raw financial metrics into key ratios and leveraging robust ensemble classification models (Random Forest), the application achieves high accuracy and reliable creditworthiness predictions. The interactive dashboard bridges the gap between complex algorithmic metrics and business decisions, making it an excellent resource for risk assessment.